



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AI-Driven Battery Saver Kernel Module

Operating System – CS235AI

Experiential Learning (Lab)

REPORT

Submitted by

Janya Gupta 1RV24CS107

Kasvi Gupta 1RV24CS125

Under the guidance of

Prof Savitri Kulkarni

Department of Computer Science

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Computer Science and Engineering

2025-2026



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled ***AI-Driven Battery Saver Kernel Module*** is carried out by **Janya Gupta (1RV24CS107)** and **Kasvi Gupta (1RV24CS125)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering (CSE)** of the **Visvesvaraya Technological University**, Belagavi during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Guide
Guide Name:

Signature of Head of the Department /
Program Coordinator

External Viva

Name of Examiners

Signature with Date

1

2



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DECLARATION

We, **Janya Gupta and Kasvi Gupta** students of Third semester B.E., department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that Experiential Learning (Lab) titled '*AI-Driven Battery Saver Kernel Module*' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering (CSE)** during the academic year 2025-26

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering®, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name

1. JANYA GUPTA (1RV24CS107)
2. KASVI GUPTA (1RV24CS125)

Signature

Abstract

Battery life is a major limitation in modern portable computing systems, especially as users increasingly rely on multitasking, background services, and resource-intensive applications. Traditional Linux power management techniques depend on static power profiles that remain fixed regardless of real-time system behavior. As a result, these methods often fail to maintain an effective balance between performance and energy efficiency when CPU usage rises or when many background processes are active. To address this issue, this project introduces an AI-driven battery saver kernel module that dynamically manages system power by continuously adapting to current workload conditions.

The proposed system continuously monitors CPU utilization and process count through a custom Linux kernel module that exposes system statistics using the `/proc` filesystem. A Python-based monitoring agent reads these values at regular intervals and forwards them to a master server that hosts a logistic regression-based machine learning model. Based on the incoming data, the model classifies the system state into one of four power modes: *Balanced*, *Moderate*, *Power Save*, and *Critical*. Each predicted mode is mapped to a predefined `nice` value, which is applied to adjust CPU scheduling priorities and control the execution of background processes.

This adaptive mechanism allows the system to automatically respond to workload fluctuations and reduce unnecessary power consumption during periods of high system stress. By integrating kernel-level monitoring, machine learning-based decision making, and process scheduling control, the proposed solution provides an intelligent and efficient method for battery optimization in Linux environments while maintaining acceptable system performance.

TABLE OF CONTENTS

ABSTRACT-----	4
CHAPTER 1- INTRODUCTION-----	6
CHAPTER 2- SOLUTION DESIGN-----	8
CHAPTER 3- IMPLEMENTATION DETAILS -----	11
CHAPTER 4- RESULTS AND DISCUSSION-----	14
CHAPTER 5- CONCLUSION AND FUTURE SCOPE -----	16
REFERENCES-----	18
APPENDIX-----	19

CHAPTER 1

INTRODUCTION

This chapter introduces the AI-Driven Battery Saver Kernel Module and highlights the need for intelligent power management in modern computing systems. With the growing use of portable devices, energy efficiency has become a critical concern, especially under heavy multitasking and high CPU usage. Traditional Linux power management tools rely on fixed power profiles that do not adapt to changing workloads. To overcome this limitation, the proposed system uses machine learning and kernel-level control to dynamically adjust CPU scheduling priorities and reduce battery consumption.

1.1 Overview of the Problem Statement

In modern operating systems, efficient power management is essential to extend battery life and ensure optimal system performance. Traditional Linux power management mechanisms operate using predefined power profiles such as Balanced, Power Saver, or Performance modes. These modes are static and require manual user selection, making them ineffective in environment where system load changes frequently. Background processes, heavy multitasking, and CPU-intensive applications cause unnecessary battery drain when they continue to run with normal priority.

The problem addressed in this project is the design and implementation of an AI-driven adaptive battery management system that responds dynamically to real-time system conditions. The system continuously monitors CPU utilization and the number of running processes to determine system stress levels. When CPU usage exceeds 50% or the process count goes beyond 250, the system identifies the condition as high load and activates battery-saving mechanisms.

The key challenge lies in integrating machine learning, kernel programming, and process scheduling into a single automated framework. The solution must accurately collect system metrics, classify system stress using a logistic regression model, and safely apply scheduling changes at the kernel level without affecting system stability or user experience.

1.2 Significance

This project demonstrates the real-world application of operating system concepts, machine learning, and kernel-level programming to solve an important problem in modern computing. By dynamically adjusting process priorities using Linux nice values, the system directly influences CPU scheduling and reduces unnecessary power consumption.

The significance of the proposed system includes improved battery life, reduced CPU overheating, and automatic performance balancing without user intervention. Unlike traditional power management tools, this system adapts continuously to workload variations. The use of a lightweight logistic regression model ensures fast decision-making with minimal overhead. The kernel-level implementation provides greater control and efficiency compared to user-space solutions, making the system suitable for energy-aware Linux environments.

1.3 Objectives

The primary objectives of this project are:

1. To design a kernel-based system to monitor real-time CPU and process activity.
2. To use a logistic regression model to predict suitable power modes.
3. To dynamically adjust process priorities using nice values.
4. To provide real-time monitoring through an agent and dashboard.

CHAPTER 2

SOLUTION DESIGN

This chapter presents the detailed design and architecture of the proposed AI-Driven Battery Saver Kernel Module. The system is divided into modular functional blocks to ensure clarity, scalability, and reliability. Each module plays a specific role in monitoring system load, predicting power modes using machine learning, and dynamically adjusting CPU scheduling priorities to reduce battery consumption.

2.1 System Architecture

The system architecture consists of the following major components, arranged in a sequential and interactive manner to ensure smooth flow of data and control:

1. System Monitoring Module

This module collects real-time system metrics, specifically CPU utilization and number of running processes. These parameters represent the current workload and stress level of the system. The collected data forms the input for the intelligent decision-making process.

2. Data Processing and Control Unit

The raw system metrics are filtered, normalized, and formatted to ensure consistent values for the prediction engine. Control logic ensures that data is sampled at regular intervals and prevents incorrect or unstable readings from being processed.

3. Machine Learning Prediction Unit

This module hosts a logistic regression model that classifies the system state into one of four power modes:

- Balanced
- Moderate
- Power Save
- Critical

The model computes a stress score using weighted inputs and determines the optimal mode based on predefined thresholds.

4. Priority Control Unit

The predicted power mode is mapped to a corresponding nice value, which controls CPU scheduling priority. Higher nice values reduce background process priority, thereby lowering CPU usage and conserving battery power.

5. Agent Communication Module

A Python-based agent acts as the communication bridge between the monitoring module, machine learning server, and priority control unit. It sends system metrics for prediction and applies the received control commands automatically.

6. Dashboard and Visualization Module

A web-based dashboard displays real-time system statistics, predicted power modes, stress levels, and historical data. It also allows manual testing of predictions for demonstration and validation purposes.

7. Overall System Operation

All modules operate continuously in a closed feedback loop. The system monitors performance, predicts optimal power states, applies scheduling changes, and observes the effect, ensuring stable and adaptive battery optimization.

2.2 Design Justification

The design choices were guided by efficiency, simplicity, and real-time adaptability. CPU utilization and process count were selected as core features because they directly indicate system workload and energy consumption. A logistic regression model was chosen due to its low computational cost, fast prediction time, and suitability for real-time systems.

Dynamic priority control using nice values allows direct influence over CPU scheduling without modifying user applications. This approach is lightweight and safe, making it ideal for kernel-level

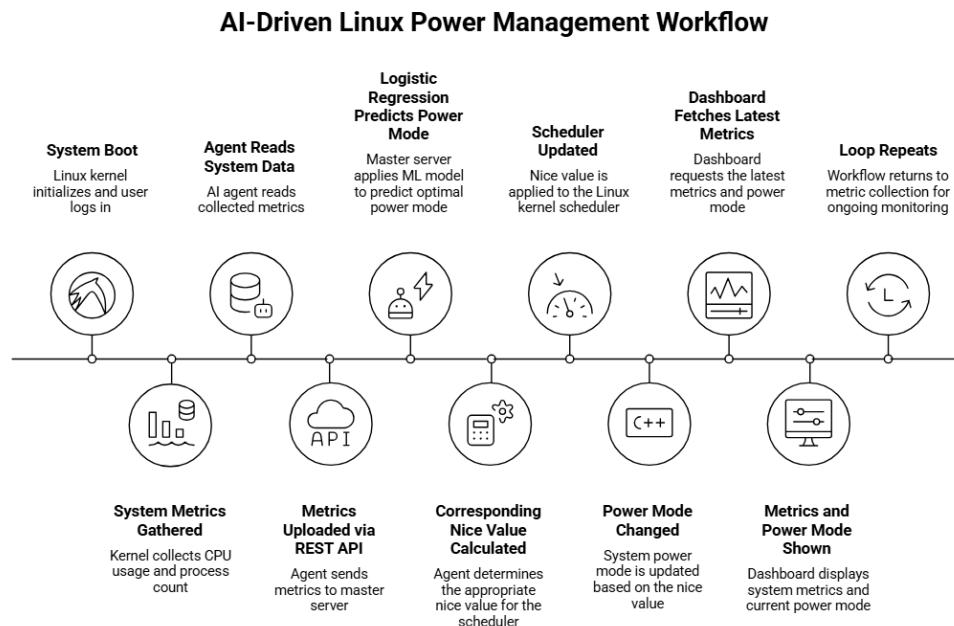
power management. Separating system monitoring, prediction, and control into independent modules ensures better reliability, scalability, and easier maintenance.

Unlike traditional static power modes, this adaptive system responds instantly to workload changes. The architecture supports future extensions such as additional features or advanced machine learning models.

2.3 Flowchart and Description

The system starts in Balanced mode. At fixed intervals, system metrics are collected and forwarded to the prediction engine. The logistic regression model evaluates the current stress level and determines the appropriate power mode. The corresponding nice value is then applied to adjust background process priorities.

If CPU usage exceeds 50% or the process count crosses 250, the system automatically shifts to a more aggressive power-saving mode. Once the load decreases, the system returns to a balanced state. This continuous loop ensures efficient battery usage while maintaining system responsiveness.



CHAPTER 3

IMPLEMENTATION DETAILS

This chapter describes the implementation approach adopted for developing the AI-Driven Battery Saver Kernel Module. The focus is on converting the proposed design into a working system using kernel programming, Python-based monitoring, and a machine learning prediction model, while ensuring modularity, clarity, and reliability.

3.1 Implementation Approach

The implementation follows a modular and layered approach, where each system component is developed and tested independently before integration. A Linux kernel module is used to access real-time system metrics, while a Python-based agent is responsible for data collection, communication, and control. A separate master server hosts the logistic regression model that performs power mode prediction.

The agent periodically reads system performance values, sends them to the prediction server, and receives the predicted power mode. Based on the result, it applies the corresponding **nice value** to dynamically adjust process priorities. This continuous feedback loop ensures automatic and adaptive power optimization without user intervention.

3.2 Design Practices

Standard software and system design practices were followed to ensure reliability and maintainability. Each module—monitoring, prediction, control, and visualization—was developed as an independent unit to allow easier testing and debugging. Clear naming conventions, structured data formats, and controlled execution cycles were used throughout the implementation.

The prediction model is lightweight to ensure fast response time and low computational overhead. System control operations are carefully gated to avoid instability and prevent unnecessary priority changes. This modular separation ensures scalability and supports future enhancements.

3.3 Project Implementation

The system is implemented using three main components:

1. A kernel module for system monitoring and control
2. A Python agent for data handling and automation

3. A machine learning server and web dashboard for prediction and visualization

The kernel module provides access to real-time system information. The agent reads these values, sends them to the server, and applies the predicted power mode. The dashboard displays CPU usage, process count, stress score, and current power mode. The complete system was tested under multiple load conditions to verify correct behavior and stability.

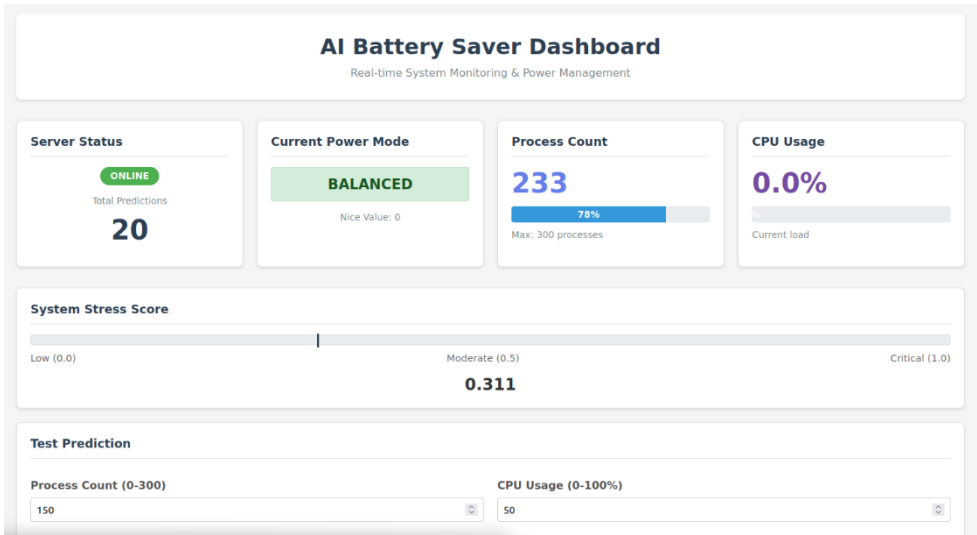


Figure 3.1: AI Battery Saver Dashboard

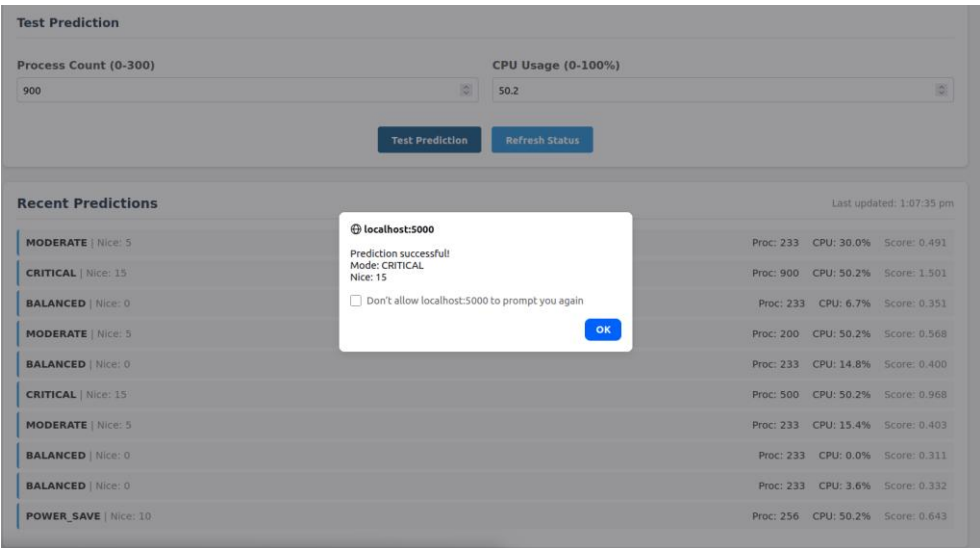


Figure 3.2: Power Mode Prediction Interface

```
AI Battery Agent Starting...
Master Server: http://localhost:5000/predict
Kernel Stats: /proc/ai_battery_stats
Kernel Control: /proc/ai_battery_control
-----
[1] Processes: 235, CPU: 9.7%
    Prediction: BALANCED (nice=0)
    Applied mode to kernel: BALANCED
[2] Processes: 235, CPU: 0.0%
    Prediction: BALANCED (nice=0)
    Applied mode to kernel: BALANCED
[3] Processes: 235, CPU: 3.3%
    Prediction: BALANCED (nice=0)
    Applied mode to kernel: BALANCED
[4] Processes: 235, CPU: 21.4%
    Prediction: MODERATE (nice=5)
    Applied mode to kernel: MODERATE

AI Battery Saver Master Server Starting...
Dashboard: http://localhost:5000
API Endpoint: http://localhost:5000/predict
* Serving Flask app 'master_server'
* Debug mode: on
WARNING: This is a development server. Do not use it
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://20.10.1.21:5000
Press CTRL+C to quit
* Restarting with stat
AI Battery Saver Master Server Starting...
Dashboard: http://localhost:5000
API Endpoint: http://localhost:5000/predict
* Debugger is active!
* Debugger PIN: 142-869-626
```

Figure 3.3: AI Battery Saver Backend Terminals

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the results obtained from testing the AI-Driven Battery Saver system and discusses its correctness, reliability, and performance.

4.1 Results

The system was successfully implemented and tested under idle, light, medium, and heavy CPU load conditions. The monitoring agent accurately captured system metrics and transmitted them to the prediction server. The logistic regression model correctly classified system states into appropriate power modes.

During high CPU usage and large process counts, the system shifted automatically to Power Save and Critical modes. When the load decreased, the system returned to Balanced mode. The dynamic adjustment of nice values reduced background CPU usage and stabilized system performance.

4.2 Discussion

The results demonstrate that the proposed AI-driven battery saver system functions reliably and responds effectively to real-time workload variations. By continuously monitoring CPU utilization and process count, the system accurately identifies changes in system stress and selects an appropriate power mode. The stress-based prediction approach produced consistent and stable results across different test conditions, showing that the model is suitable for real-time operation.

Dynamic adjustment of process priorities using nice values proved to be an efficient way to reduce unnecessary CPU usage during high load conditions. Background tasks were successfully throttled while essential processes continued to run smoothly, maintaining system responsiveness. The modular structure of the system ensured smooth communication between components, allowing the system to operate continuously and making it easy to extend for future improvements.

4.3 Test Case Validation

Test Case	CPU Usage	Process Control	Expected outcome	Result
1	Low (15%)	180	Balanced	Pass
2	Medium (35%)	210	Moderate	Pass
3	High (60%)	260	Power Save	Pass
4	Very High (95%)	290	Critical	Pass

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

This chapter summarizes the overall outcomes of the project and discusses possible future enhancements to improve functionality, scalability, and real-world applicability of the proposed system.

5.1 Conclusion

The AI-Driven Battery Saver Kernel Module successfully demonstrates an intelligent and adaptive power management solution for Linux-based systems. By integrating real-time system monitoring with a machine learning-based decision model, the project highlights an effective approach to reducing battery consumption under varying workload conditions. Unlike traditional static power management tools, the proposed system dynamically adjusts CPU scheduling priorities based on current system stress, ensuring efficient energy usage without manual intervention.

The use of a logistic regression model enabled fast and lightweight prediction of optimal power modes using CPU utilization and process count as input parameters. Dynamic priority control through nice values proved to be an effective mechanism for throttling background processes during high-load conditions while maintaining system responsiveness for active tasks. The modular architecture ensured reliable communication between monitoring, prediction, and control components.

Key outcomes:

1. Real-time detection of system workload using CPU and process metrics
2. Accurate prediction of power modes using a machine learning model
3. Dynamic adjustment of process priorities to conserve battery power
4. Stable and automated operation without affecting system usability

5.2 Future Scope

The proposed system provides a strong foundation that can be extended to support more advanced and real-world power management features. With additional parameters and enhancements, the system can be made more intelligent and robust.

Possible enhancements:

- Incorporate battery percentage,
- temperature, and power consumption data as additional inputs
- Replace logistic regression with advanced machine learning or deep learning models
- Integrate with ACPI and hardware-level power management features
- Extend support to mobile and embedded Linux platforms
- Add user-configurable profiles and mobile dashboard access

REFERENCES

- [1] A. Carroll and G. Heiser, “An analysis of power consumption in a smartphone,” *USENIX Annual Technical Conference*, 2010.
- [2] M. Pedram and J. Rabaey, *Power Aware Design Methodologies*, Springer, 2002.
- [3] S. Mittal, “A survey of techniques for improving energy efficiency in embedded computing systems,” *International Journal of Computer Aided Engineering and Technology*, 2014.
- [4] Linux Kernel Documentation – Process Scheduling and Nice Values.
- [5] Scikit-learn Documentation – Logistic Regression Models.

APPENDIX

Appendix A: System Architecture and Dashboard Screenshots

The complete system architecture diagram, kernel module interaction, and dashboard screenshots demonstrating real-time CPU usage, process count, stress score, and power mode prediction are provided in Chapter 3: Implementation Details.

Appendix B: Flow Description

The flow describing system monitoring, machine learning prediction, and dynamic priority control is included in Chapter 2: Solution Design.

Appendix C: Test Cases

The test cases validating system behavior under idle, light, medium, and heavy load conditions are discussed in Chapter 4: Results and Discussion.